Linux Device Update - Part 1

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

Learning objectives:

Understand common design goals for Embedded Linux Device Update

Device Update Overview

Development Environment

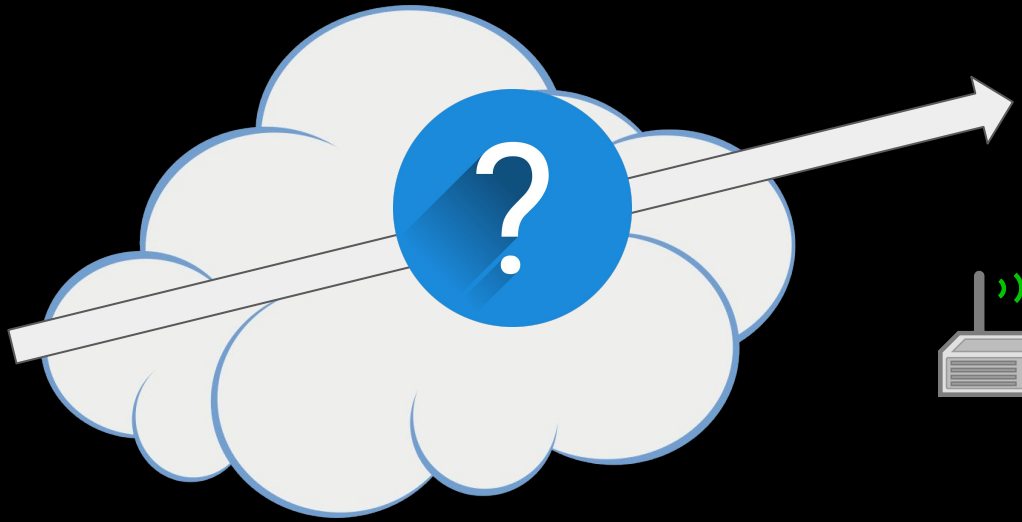


Device Update File

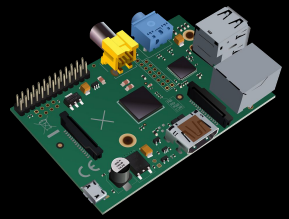




Cloud Server



Device





Wireless Router

OTA (Over The Air) Updates

- Updating software on Embedded Devices after shipment
- Commonly referred to as “OTA” for wireless applications
- Security
  - Asymmetric Cryptography
- Reliability
  - A/B Update Scheme
- Ease of Use
  - Transport considerations

Platform Support/Tradeoffs

More Hardware Platform/Service Cost Less



Particle









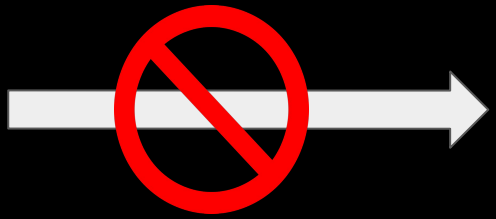
More Development Effort Required Less

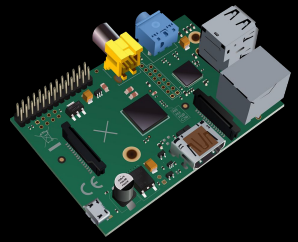
Designing your Device Update Solution

- Security
  - Device update can be a backdoor
  - Obfuscation != Cryptography



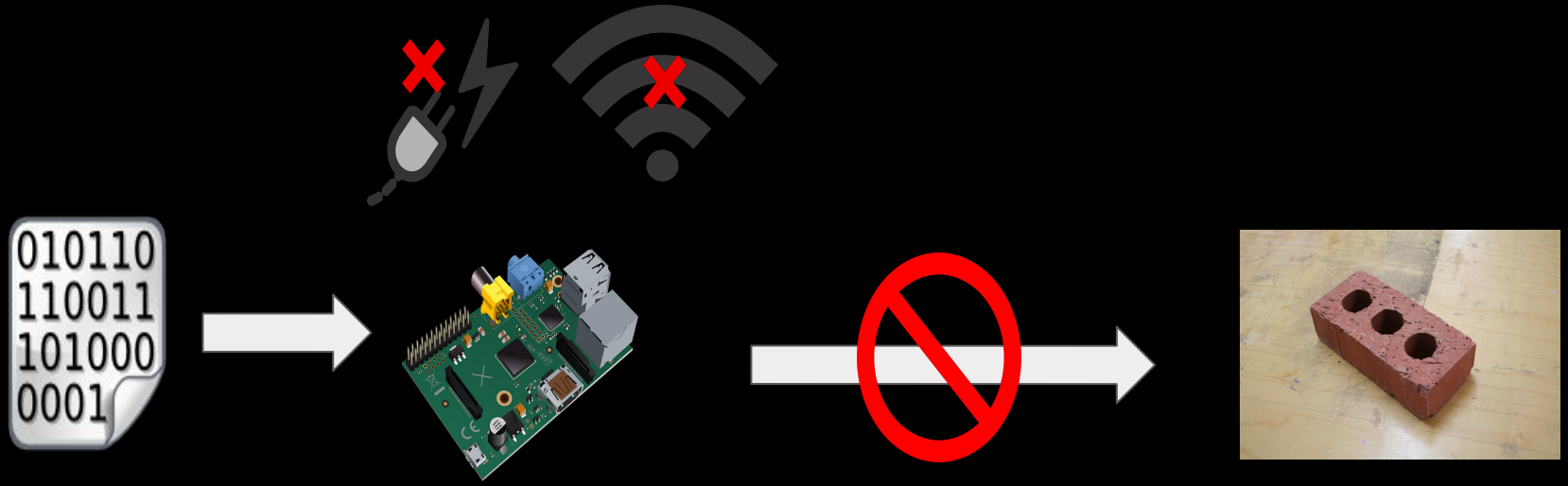






Designing your Device Update Solution

- Reliability



Designing your Device Update Solution

- Ease of Use (for end users as well as developers)

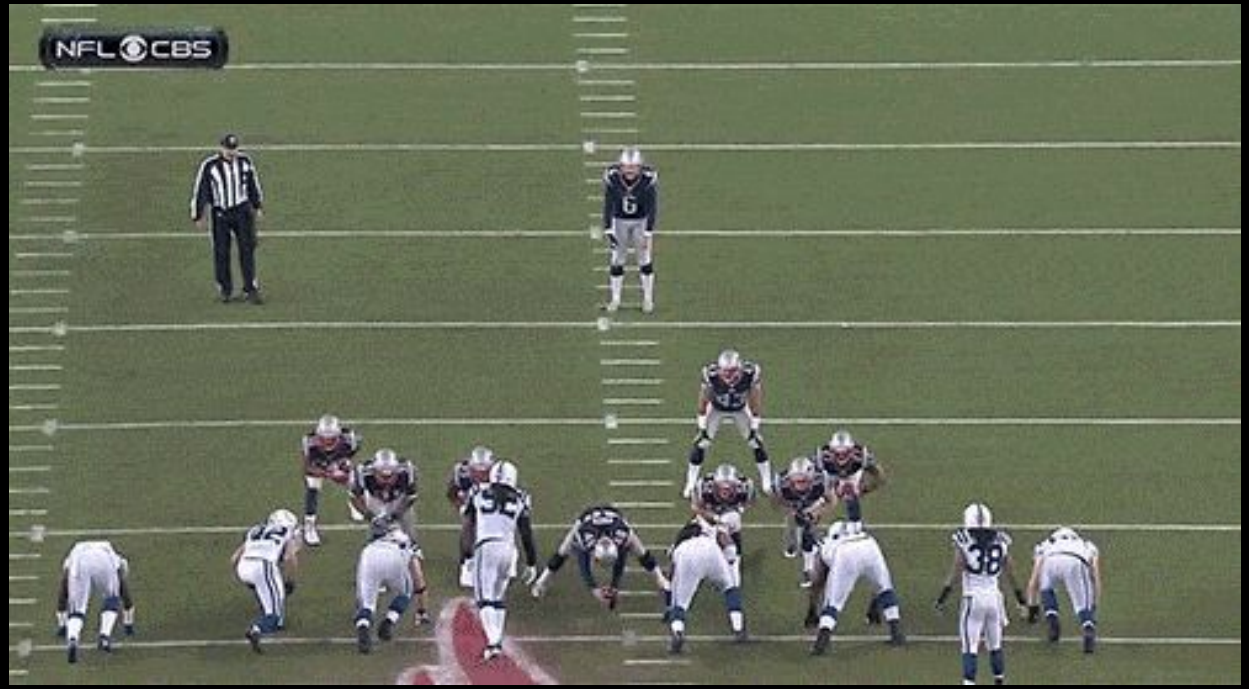




Designing your Device Update Solution

- Great device update solutions are invisible, valued for what they don't do
  - Don't expose your end users or your company to hackers
  - Don't turn your devices into bricks
  - Don't burden your customers
  - Don't burden your developers
- Football equivalent of a long snapper

Designing your Device Update Solution



https://giphy.com/gifs/new-england-patriots-XXiXTjaz10axG

Linux Device Update - Part 2

Advanced Embedded Software Development

with **Dan Walkes**



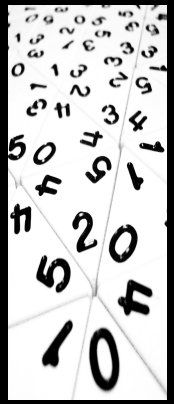
University of Colorado **Boulder**

Learning objectives:

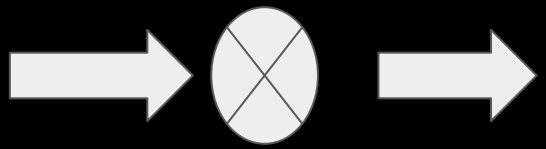
Understand the function and role of cryptographic signing with Device Update

Signing: Key Generation

Random Number Generator



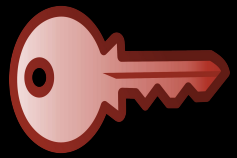
Key Generation Algorithm (RSA/ECC)



Public Key



Private Key



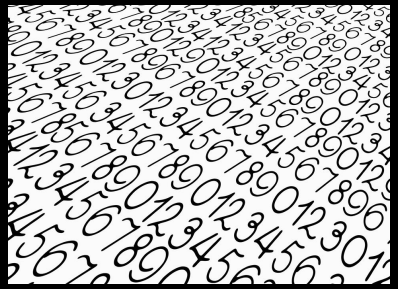
Very large numbers

- 256 bits or 77 decimal digits
- Special mathematical properties
- Approximately large enough to count the number of atoms in the universe.
- Not possible or practical to guess or try every value
    - Fastest supercomputer would take multiple universe lifetimes just to try simple math on each possible value

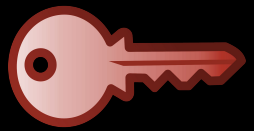
Signing: Signature Generation

Very large number

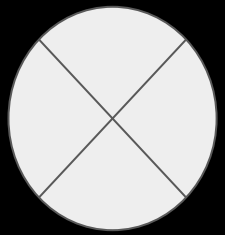
- Specific length (256 bits or 77 decimal digits)



Private Key



Signature/Encryption Algorithm (RSA or ECC)



Signature (Large Number)



Signing: Signature Validation

Very large number
- Specific length (256 bits or 77 decimal digits)



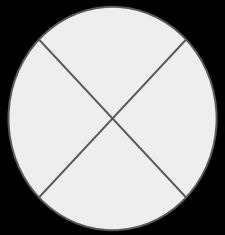
Public Key





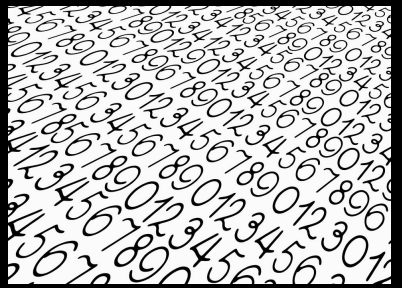


Signature/Encryption Algorithm (RSA or ECC)

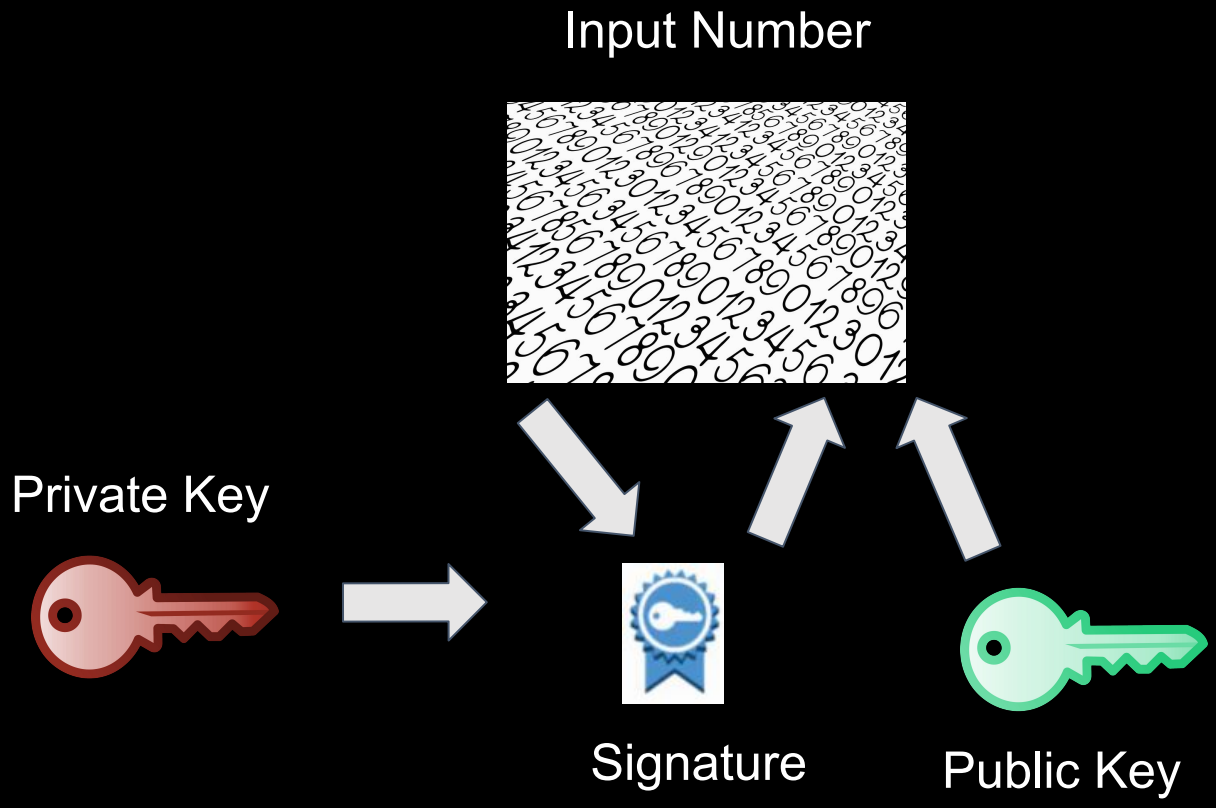




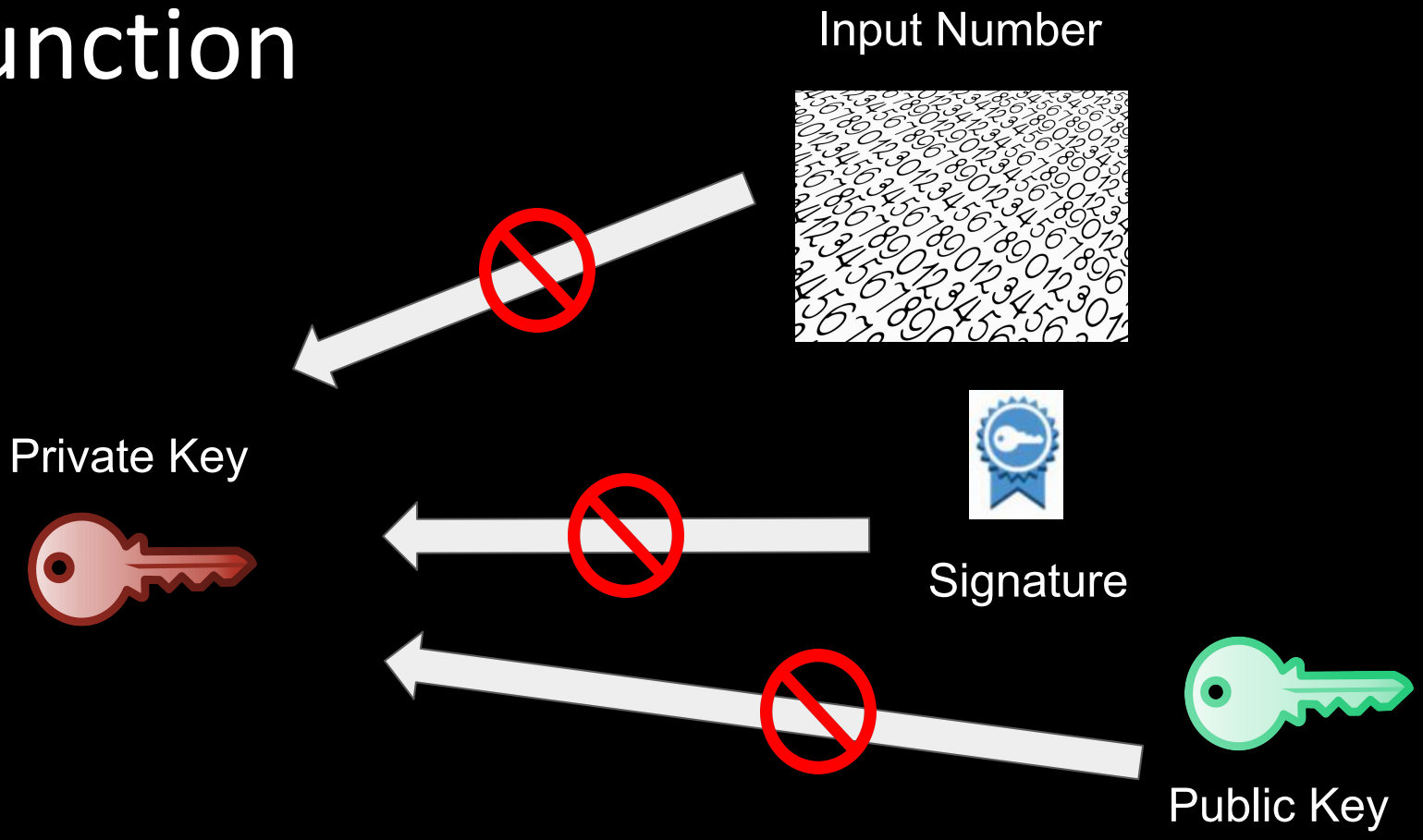
Same large number input with Private Key at Signature Generation step



Security Overview: Relationships



Security Overview: One-way Function



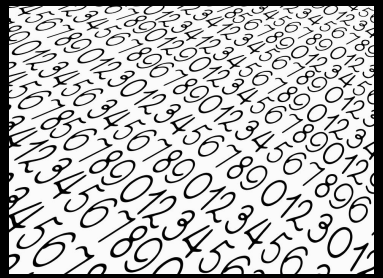
Security Overview: Hash

Binary File of any length



Hash Function (SHA)

Number of specific length

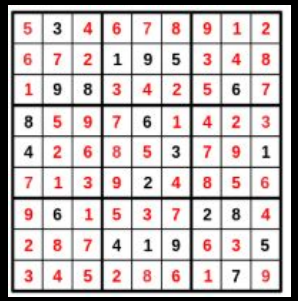


Slightly modified Binary File



Hash Function (SHA)

Completely different number (but same length)



Security: Key Storage





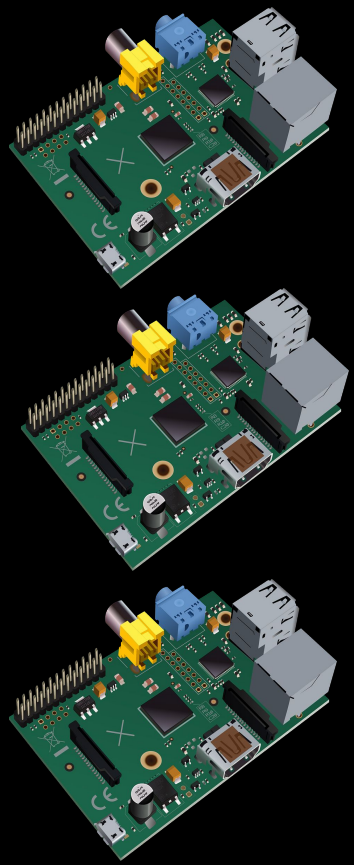
Private Key

- Never leaves the development environment

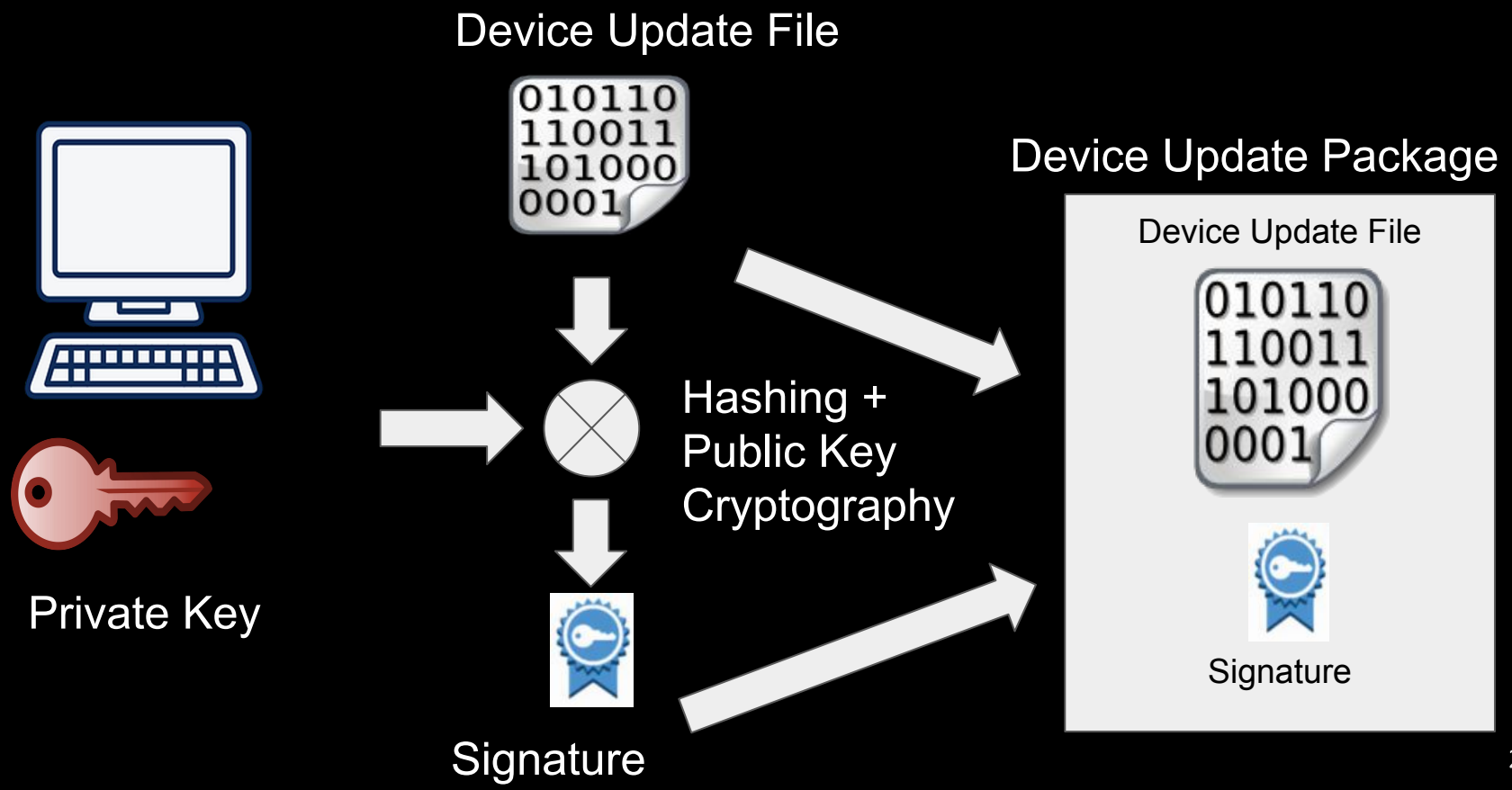


Public Key

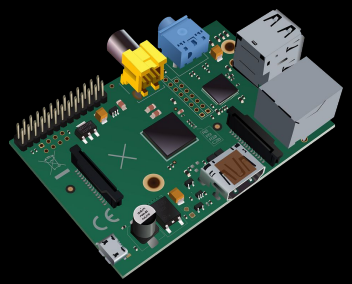
- Saved on each device



Creating Signed Update File



Verifying Signed Update File



Public Key

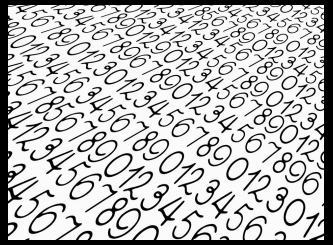




Signature

RSA or ECC

Signed Hash

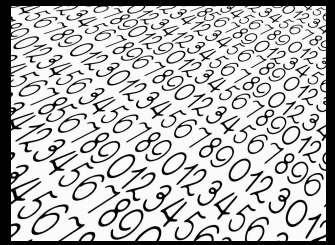


Device Update File



SHA

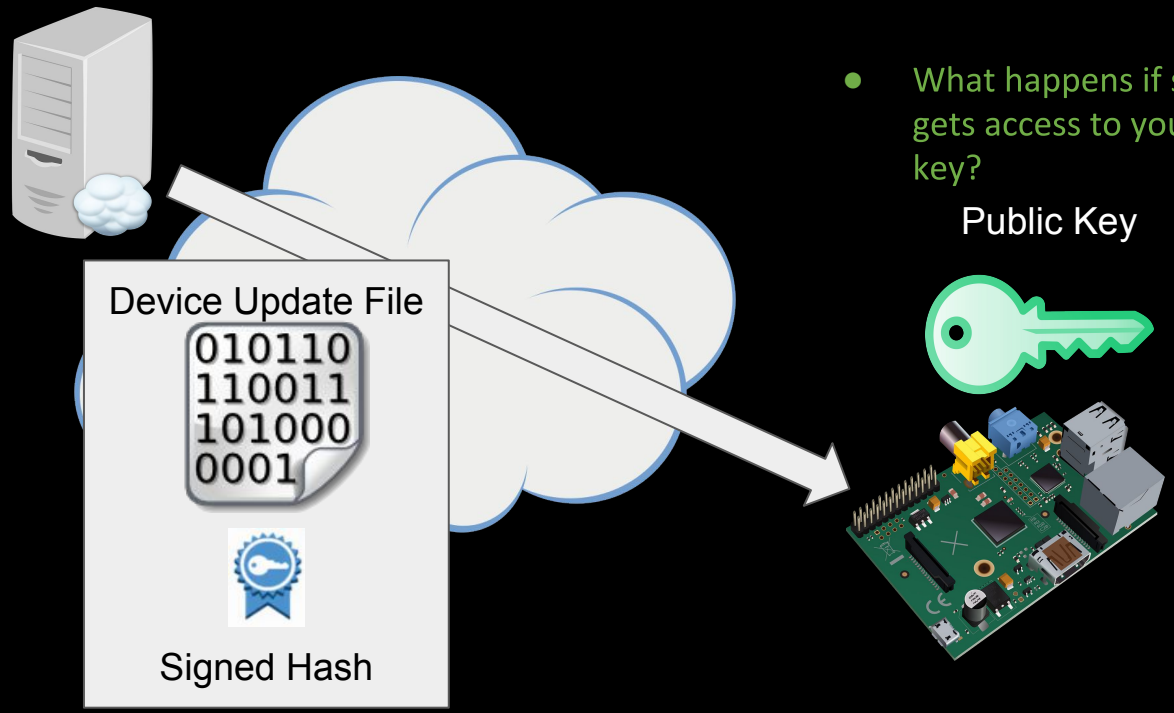
Computed Hash



Update Verified only if Signed Hash = Computed Hash

Verifying Signed Update File

- What happens if someone hacks your cloud server or file transport?



- What happens if someone gets access to your public key?

Signing Schemes and Quantum Computing

- RSA and ECC schemes make assumptions about capabilities of existing hardware platforms with regard to factoring prime numbers
- The rules will change with the advent of Quantum Computing
- Process will require thousands of “entangled qbits”
- Currently able to entangle on the order of 20 qbits (IBM Poughkeepsie Sept 2019)
- Somewhere between 5 and 15 years this may be possible with billion dollar supercomputers
- Active NIST program targeting new quantum resistant standards in the 2022-2024 timeframe

https://techbeacon.com/security/waiting-quantum-computing-why-encryption-has-nothing-worry-about
https://www.nature.com/articles/s41598-019-49805-7
https://csrc.nist.gov/CSRC/media/Presentations/Update-on-the-NIST-Post-Quantum-Cryptography-Proje/images-media/2_post-quantum_dmoody.pdf
https://csrc.nist.gov/Projects/post-quantum-cryptography/workshops-and-timeline

Linux Device Update - Part 3

Advanced Embedded Software Development

with **Dan Walkes**



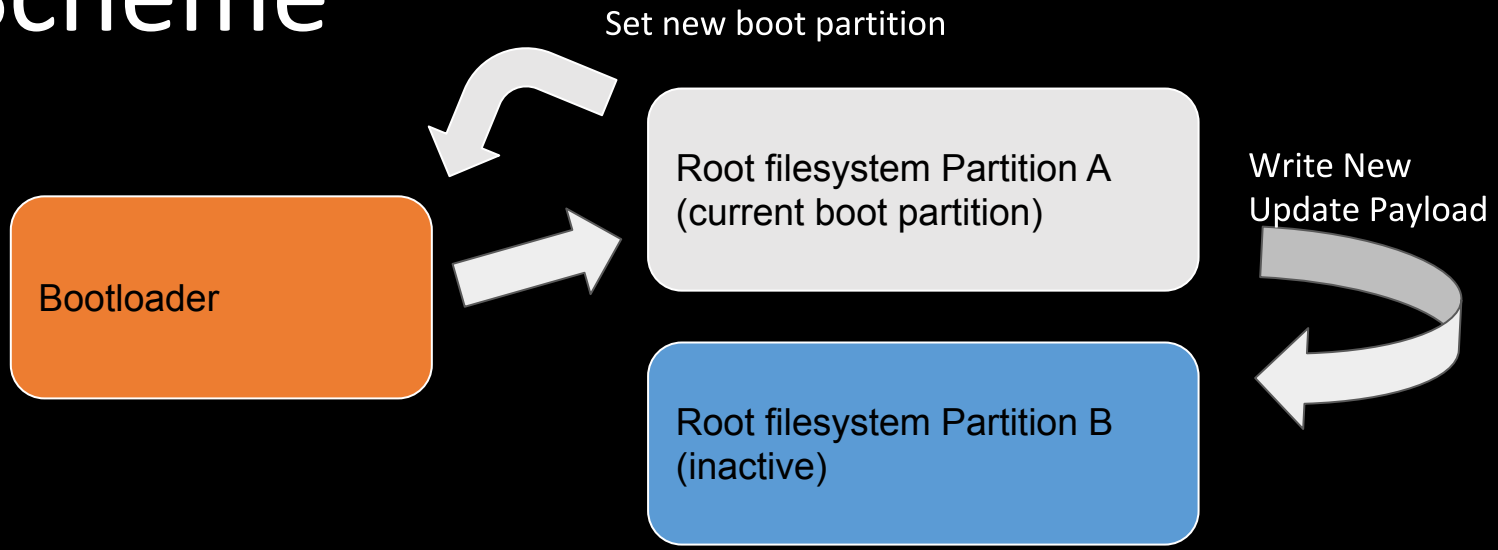
University of Colorado **Boulder**

Learning objectives:

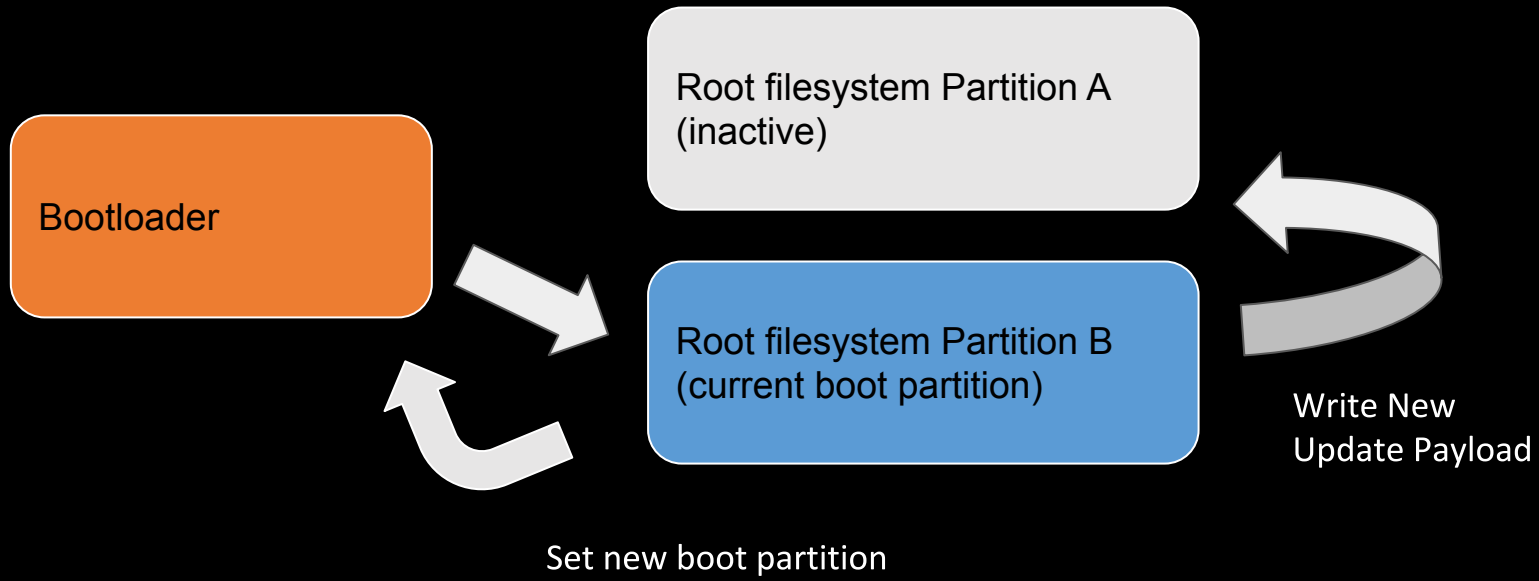
Understand Embedded Linux device update strategies and tradeoffs.

Introduce open source frameworks for device update

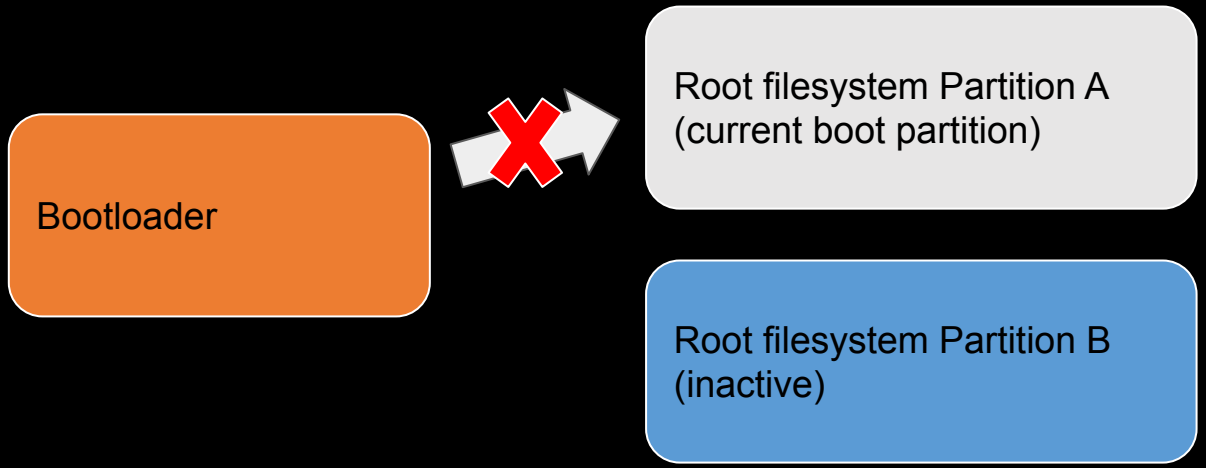
A/B or Dual Redundant Update Scheme



A/B or Dual Redundant Update Scheme

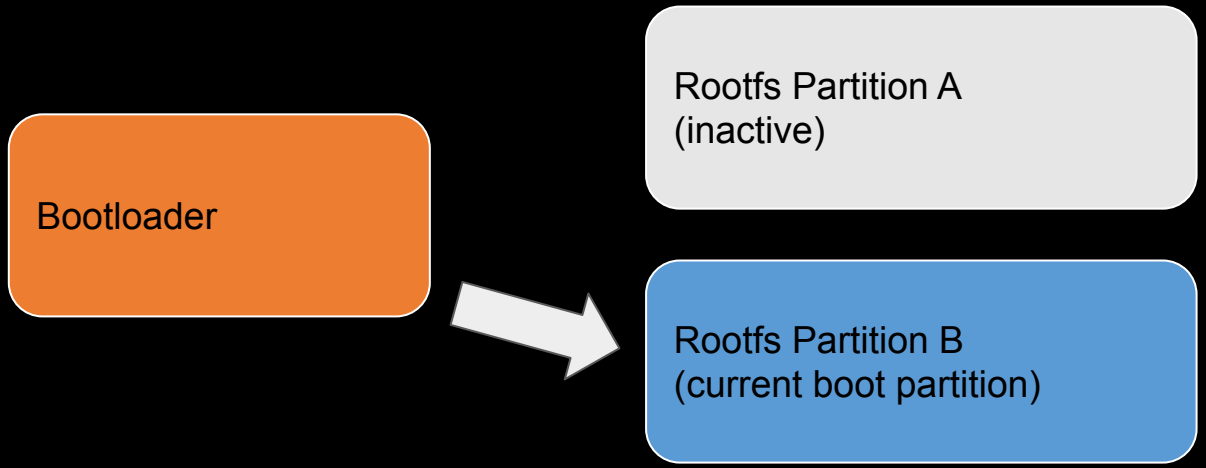


A/B or Dual Redundant Update Scheme



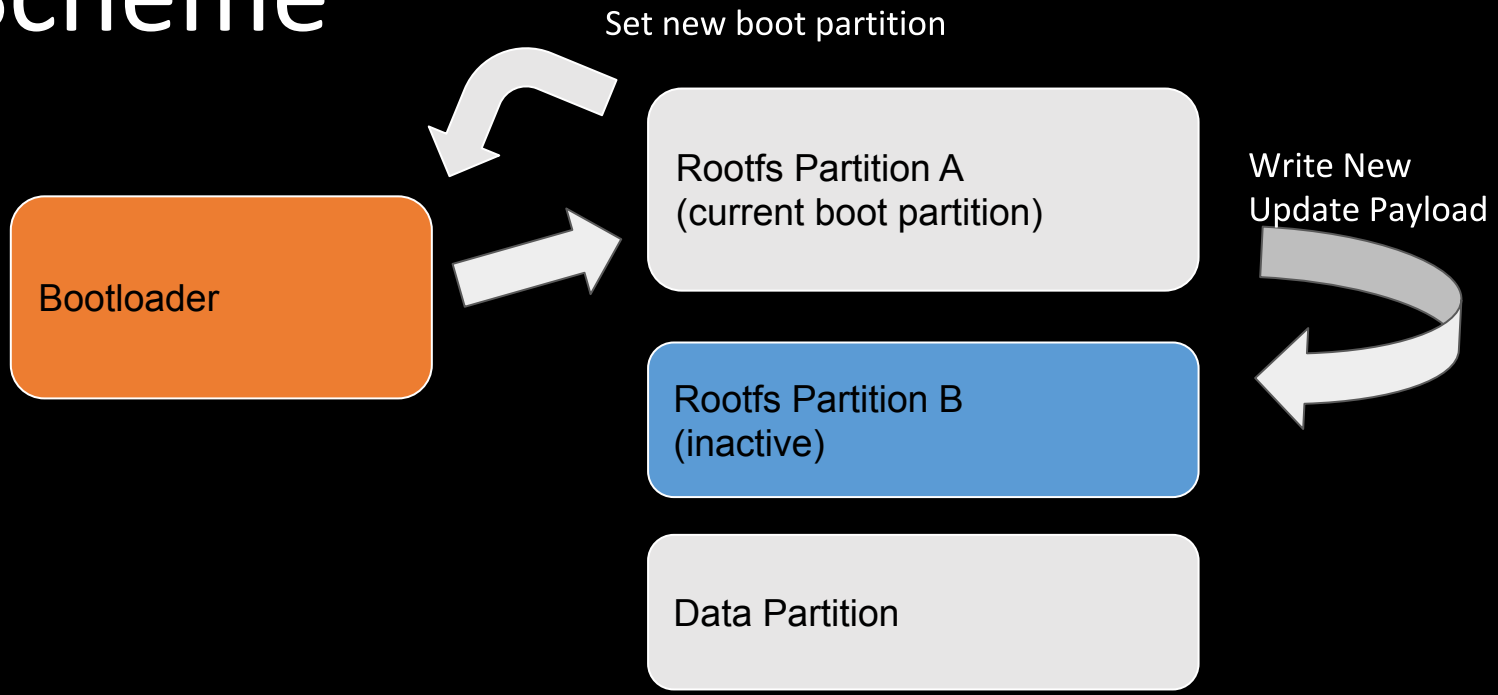
- What if boot fails?

A/B or Dual Redundant Update Scheme



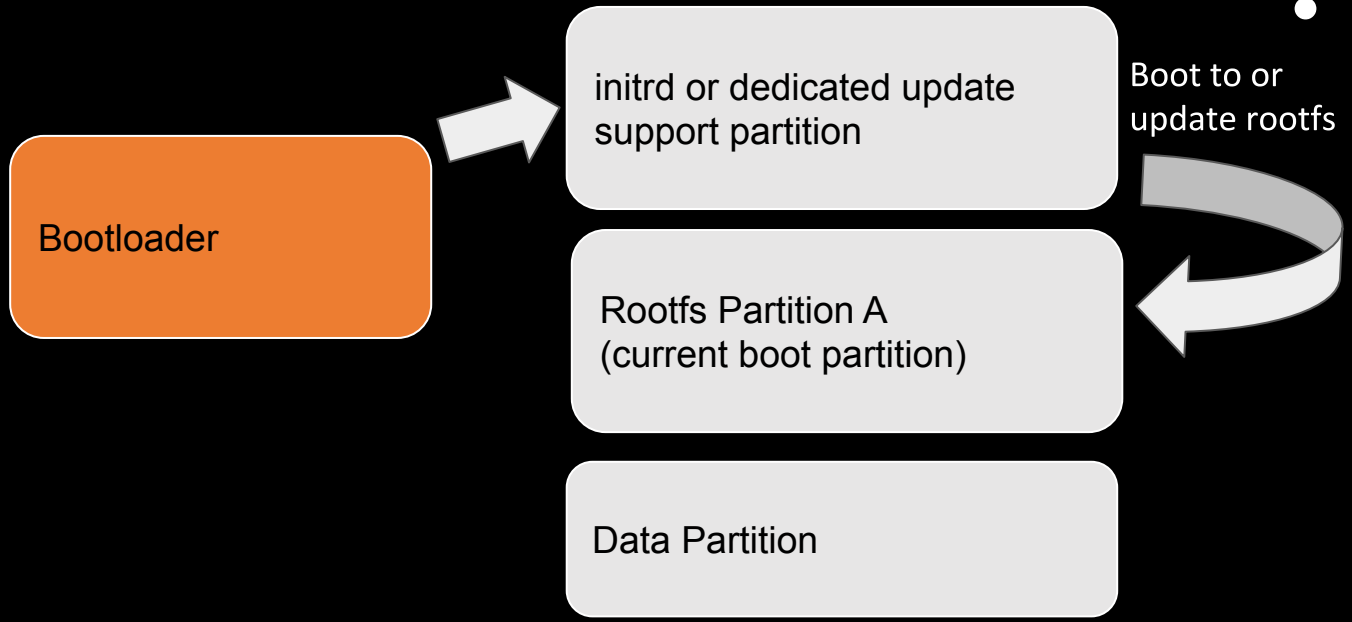
- What if boot fails?
  - Bootloader can fall back to previous boot
  - Based on state set in image when boot succeeds

A/B or Dual Redundant Update Scheme



- What about configuration data modified on the root filesystem?
  - Need to keep in another non-modified partition

Single Copy



- Initrd contains minimal logic to:
  - Detect whether rootfs partition is bootable
  - Support upgrades in non-bootable cases

- Alternative to A/B Update Scheme: Use a single rootfs but put update logic (at least recovery logic) in initrd image.

https://sbabic.github.io/swupdate/overview.html#single-copy

Application Update Scheme

- Instead of updating the entire root filesystem, update only specific applications.
- Applications are updated with packages like .deb and .rpm along with user space applications like apt/yum
  - Replace components associated with each application including
    - binaries
    - start/run scripts
    - configuration files
- Update packages are built during yocto image build
- This scheme is used for standard Linux distribution upgrades (like debian, Ubuntu,etc)

Application Update vs A/B Update

- A/B Update Benefits
  - Able to completely roll back to a previous working or shipped release (possibly automatically)
  - Don’t need to worry about partial upgrades/different application update states.
  - Logical separation between configuration data and update image.
    - Easier to handle factory reset scenarios
- A/B Update Drawbacks
  - Requires double size of storage for rootfs.
  - Requires full (compressed) root update size for each image (or more complicated delta update scheme).
- Application Update Scheme Benefits
  - Smaller update sizes, no additional storage space requirements
- Application Update Scheme Drawbacks
  - Rollback is more complicated
  - May have inconsistent states during power cycle

Update Schemes Compared

Criteria	A/B Dual Redundant	Single Copy	Application Update
Storage Size Requirements	Most (2x root partition size)	1x root partition size plus storage space for upgrade components	No/minimal space required
Redundancy and rollback Capabilities	Best (all components including update components can be rolled back)	Good (most components are redundant, update components may be difficult or not possible to update redundantly)	Most limited (rollback may be complicated, images may be inconsistent)
System Complexity	Higher (extra data partition, bootloader redundancy)	Higher (extra data partition, initrd/update setup)	Lowest (no extra data partition)

Linux Device Update Frameworks: swupdate

- “Building block” for upgrades
  - Used with specific upgrade designs
  - Doesn’t enforce or provide a specific update strategy
- Supports Buildroot or Yocto
- Supports A/B, Single Copy or Application Update schemes
- Supports signing
- Integrates with Eclipse Hawkbit for cloud deployment scenarios
- Supports an integrated web server for deploying images



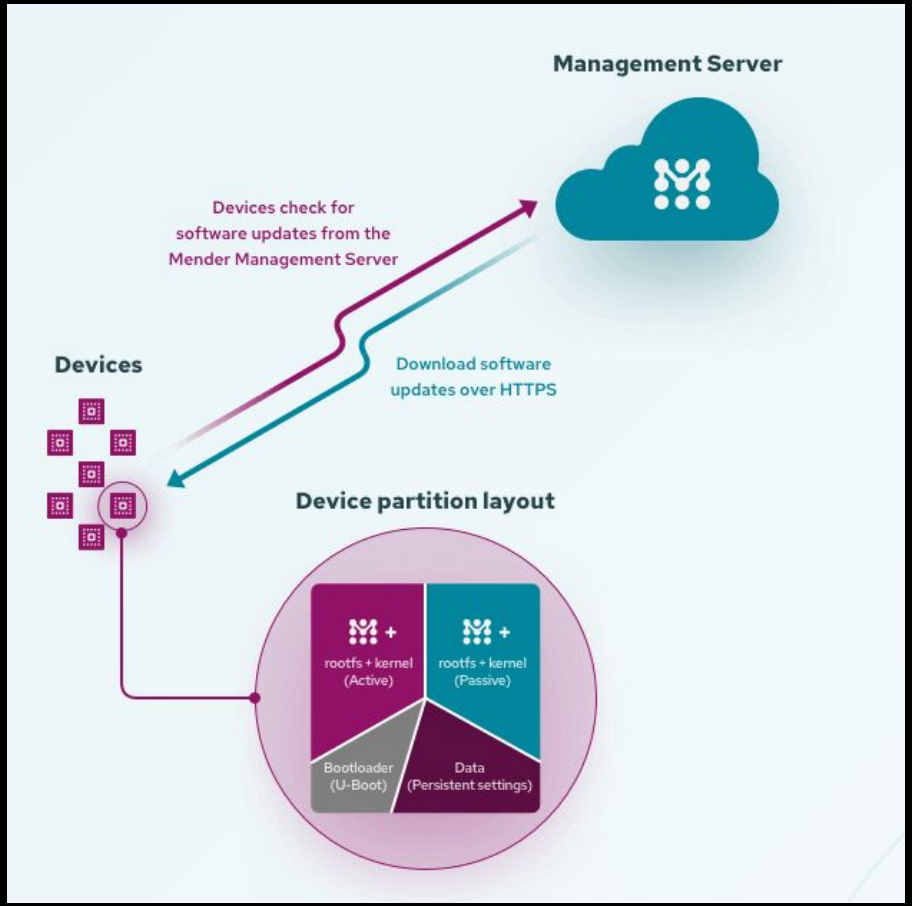
https://sbabic.github.io/swupdate/

Linux Device Update Frameworks: mender

- Built around Yocto
  - Includes layers providing u-boot modifications
- Supports buildroot
- Built around A/B Dual Redundant Image support
  - Also supports Application Updates on some platforms as of 2.0
- Supports signing
- Supports delta updates
- Supports cloud deployment through integrated open source Mender Management server
  - Also available in hosted form

https://mender.io/blog/mender-2-0-released-application-updates
https://mender.io/overview/how-it-works

Linux Device Update Frameworks: mender



https://mender.io/overview/how-it-works